

Machine learning in production

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 1, `nw/triage`: a calibrated priority classifier that reaches production only through a gate, a pipeline, a registry and a person's approval. Then the service that serves it on your track, measures drift on every request, shadows the next version, and a retraining job that never promotes on its own.

Supervised learning predicts a label; unsupervised learning finds structure

Supervised

Learns to predict explicit, often human-given labels.

At Northwind: priority, learned from 7,950 tickets someone already labelled. That is Project 1.

Also: fraud, churn, credit risk, demand

Unsupervised

Learns patterns in data nobody labelled, for example clusters.

At Northwind: grouping this week's tickets to find a topic no tag covers yet.

Also: segmentation, anomaly detection

"Classical" means the models before deep learning: linear models, trees, gradient boosting. They are still the default for tabular data and narrow text tasks.

A small model beats an LLM when the task is narrow and the volume is high

Latency

Topic classification: a fine-tuned encoder took about 0.02 s per text; most LLMs 0.6 to 1.4 s; GPT-5 5.5 s.

Control

Fine-tuned BERT-style models beat zero-shot GPT-4 and Claude Opus on every classification task tested.

Cost

No tokens: a prediction is CPU time on a service you already run. Project 1 answers in 1.7 ms at the median.

Audit

US lenders must give specific reasons for a credit denial: "there is no special exemption for artificial intelligence".

Small, purpose-built models still run the decisions that need milliseconds and a reason

Where	What the evidence says
Tabular competitions, 2025	XGBoost and LightGBM each in 14 winning solutions, CatBoost in 8
Uber's platform, 2024	over 5,000 production models, 10 million predictions a second at peak; "in several cases, XGBoost outperforms DL"
Stripe Radar	each payment scored in under 100 ms on over 1,000 signals; 0.1 percent wrongly blocked
Credit scoring, EU	high-risk under the AI Act; fraud detection is explicitly excluded
Benchmarks, 2025 and 2026	trees stay competitive on tabular data; tabular foundation models lead on small data; no family wins everywhere

MLOps is a maturity ladder, and most teams stand on its bottom rung

Level	What it means	In Project 1
0	manual, script-driven, infrequent releases	where a notebook and a copied model file stop
1	an automated pipeline, continuous training, data and model validation	the data contract, the pipeline, the gate, the registry, the paused retraining job
2	automated CI/CD for the pipeline itself	<code>retrain-triage.yml</code> gates pull requests; the capstone's delivery pipeline

"MLOps level 0 is common in many businesses." Google Cloud Architecture Center

Only a small fraction of a real-world ML system is the ML code. Today writes that fraction in the first hour, and the rest of the system in the second.

Where this sits: the service layer gave every model call one door, and Project 1 needs none



Every part asks which kind of AI fits the job. For ordering Northwind's queue by priority, the answer is a linear model on word and character features, trained in seconds. The agent calls it as a tool; the capstone puts it in front of the agent as a cheap pre-filter and promotes it behind a canary.

Today you build a triage model that only a person can promote

The model

A calibrated classifier with a threshold that protects P0 recall: 0.581 in the pre-work, 0.903 by the gate.

The loop

A data contract, a gate, a pipeline and a registry, the same on every track.

The person

Approves with a reason. The service serves only what was approved, and watches for drift.

A pipeline can propose. Only a person promotes.

Today, step by step

- 1 Read the data and the contract
- 2 The naive model meets the gate
- 3 Class weights
- 4 A threshold, not argmax
- 5 Calibrate the probabilities
- 6 Track and compare runs
- 7 Train through the pipeline
- 8 Read the model card and the promotion log
- 9 Read the registry
- 10 Approve the candidate and see it served
- 11 Alive is not ready; drift
- 12 Shadow the next version **OPTIONAL**
- 13 The platform's monitor
- 14 The container and the packaging **OPTIONAL**
- 15 The retraining loop, paused **OPTIONAL**
- 16 Check the part and commit

Step 7 submits a cloud run that takes minutes. Do not wait: steps 8 and 9 read the card and then the result.

Accuracy is a trap on four percent P0

3.9%

of Northwind's tickets are P0. A model that never predicts P0 is 96 percent accurate on that class and useless

Metric	Why it matters here
Accuracy	dominated by the common classes; hides P0
Macro - F1	rewards getting the rare class right
P0 recall	the commercial number: a missed P0 is an SLA breach
P0 precision	the cost of the recall you buy
Brier and ECE	whether a threshold on the probability means anything

Nothing trains until the data passes its contract

Expectation	What it catches
Every row matches the pydantic contract	a renamed column, an empty body
P0 between 2 and 8 percent	an export that drops or floods the rare class
Validation and test at least 8 percent	numbers that rest on too few rows
Duplicates under 0.5 percent, no split leak	a test number that is a lie
Every label in every split	a sweep that cannot run

```
make data-check
```

rows 7950, splits 6361 / 795 / 794, P0 3.9%, de 4.9%, duplicates 0, **DATA OK**.

The same run writes the data profile. It ships inside every artifact, and the pipeline's first step runs the same check.

The naive model sends 13 real emergencies to the ordinary queue

18 of 31

true P0 tickets caught by argmax on the test split. The P0 row reads 18, 13, 0, 0

- Accuracy 0.694, macro-F1 0.688 against a majority baseline of 0.414 and 0.146
- P0 precision 0.900: usually right when it says P0, the wrong trade for an outage
- **GATE FAILED**: P0 recall 0.581 below the bar 0.85, and more than 0.03 below production's 0.903
- **make session02**: 5 failed, 19 passed, 1 skipped, 10 errors

The same refusal the pre-work pipeline printed at its register step. A model can pass most of a test suite and still be useless for the one thing the business needs.

A candidate goes anywhere only by passing the gate

Bar	Value	What it protects
P0 recall, absolute	at least 0.85	Northwind's SLA, whatever the last model did
ECE, absolute	at most 0.12	the threshold means something
Macro-F1 against production	drop of at most 0.02	recall not bought by wrecking three classes
P0 recall against production	drop of at most 0.03	no quiet regression on the commercial number
Brier against production	increase of at most 0.01	probabilities not getting worse
Same test split	identical data hash	two numbers that mean the same thing

On the laptop a pass moves `latest`. In the pipeline it moves nothing: the register step proposes the candidate, and a person decides.

Three fixes, cheapest first, and only the third gets through

Step	Fix	Threshold	Macro-F1	P0 recall	P0 precision	The gate says
2	none	0.50	0.688	0.581	0.900	recall below the bar
3	class weights	0.50	0.717	0.774	0.828	recall below the bar
4	a threshold chosen on validation	0.05	0.635	0.935	0.377	macro-F1 drops more than 0.02
5	sigmoid calibration	0.05	0.663	0.903	0.467	PROMOTED

Never on the test split. The sweep runs on validation. The test split is the number you report; choosing on it makes that number a lie.

Calibration made two numbers worse and the decision better

28 of 31

P0 caught at threshold 0.05 after calibration. False P0 alarms from the P1 row: 38 down to 28

- Brier (P0) 0.0115 to 0.0140, ECE 0.058 to 0.091: slightly worse, and the report says so
- Argmax P0 recall falls to 0.677: the sigmoid map pulls raw P0 scores towards the truth, and P0 is rare
- At 0.05, ten P1 tickets drop below the line and the gate passes

A run is not a promotion

weights	calibrated	threshold	macro-F1	P0 recall	P0 precision	
none	no	0.50	0.688	0.581	0.900	
balanced	no	0.50	0.717	0.774	0.828	
balanced	no	0.05	0.635	0.935	0.377	
balanced	yes	0.05	0.663	0.903	0.467	latest
balanced	yes, floor 0.5	0.09	0.687	0.903	0.560	

The fifth row would pass the gate. It is not served because nobody decided it should be: a precision floor is the on-call lead's number, not yours.

The pipeline is your code, cut into four plain steps

Step	Writes
data check	the profile; exit 1 on a blocking finding
train	the versioned candidate, the run record, the card
evaluate	the gate's decision in <code>gate.json</code>
register	the candidate in your registry, only if the gate passed

The steps are functions in `nw/pipelines/steps/`. Only the definition differs per track.

One definition, two runners, one second SDK

Local and Google Cloud: the same compiled Kubeflow YAML, on the local runner and on Pipelines.

AWS: a SageMaker definition over the same step code, with a ConditionStep on the gate's `passed_int` and a FailStep that carries its reasons.

The card says what the report did not

1 of 2

German P0 tickets caught in the test split. 39 German rows, macro-F1 0.555 against 0.668 for English

`MODEL_CARD.md` is generated from the artifact, so it cannot go stale:

- intended use; the model, threshold and targets; the data hash
- the test table, then the same numbers per language
- limitations, and how it is monitored

`promotions.jsonl` beside it: every gate decision, and `"forced": true` when someone overrode one.

A registry version is an artifact, its measurements and a stage

Stage	Who moves it	What it means
candidate	the pipeline's register step	cleared the gate; nobody has read it yet
approved	a reviewer, <code>set_stage(..., Stage.APPROVED, reason)</code>	served on your tenant's endpoint
live	the capstone's promotion drill, with a canary	served to customers on the live target
retired	the next <code>live</code>	kept for rollback and audit

The artifact is immutable. The stage is the only thing that changes, and it changes by a person, with a sentence.

Approval is the same call on four platforms

AWS

The package turns `Approved` ; the state change event reaches `northwind-deploy-on-approval` , which creates your Serverless Inference endpoint within a minute. Solo mode deploys it directly.

Google Cloud

The alias moves to `approved` with your reason in `stages.jsonl` .
`endpoints.deploy` rewrites `NW_MODEL_URI` on your Cloud Run service: a new revision reading the approved artifact.

Azure

The `stage` tag on the Azure Machine Learning model moves to `approved` , reason in `stages.jsonl` .
`endpoints.deploy` puts it on your managed online endpoint.

Local

The MLflow alias and the `stage_reason` tag.
`deploy(live=True)` restarts `serving-stable` on the registry's artifact; the proxy on :8005 reads `stable 100, canary 0` .

`approved` is not `live` . What you deployed is your tenant's endpoint, which nobody but you calls. Customers see a model only through the capstone's canary.

A model that fails to load leaves the process alive and not ready

Endpoint	Who calls it
<code>/healthz</code>	the liveness probe; failing it restarts
<code>/readyz</code>	the load balancer; failing it stops traffic
<code>/trriage</code>	the agent, the capstone router
<code>/metrics</code> , <code>/version</code> , <code>/drift</code>	dashboards, "which model", "which world"

On the platform the service reads `NW_MODEL_URI` from the registry, not `latest`.

Load fails

Process stays up, `ready = False`, one log line with the path or URI and the error.
`/healthz` 200, `/readyz` and `/trriage` 503.

Crashing loops the restart and scrolls the one useful line off the screen.

Two watchers, one question: is this still the world it was trained on

8.87

text-length PSI after fifty one-line tickets. Below 0.1 stable, 0.1 to 0.2 watch, above 0.2 alert

The service, per request, in milliseconds: PSI of text length and predicted priority over the last 500 requests, one `drift_alert` line past 0.2.

The platform, on a schedule, from outside: Model Monitor, Model Monitoring or Evidently reading the captured traffic.

AWS closed Model Monitor to new customers. On an account that never used it the schedule fails to create and the endpoint still serves; the service's `drift_alert` metric filter is the alarm that fires.

The next version runs beside the served one first

A candidate trained for recall 0.95 fell back to threshold 0.48, silently. Twenty real P0 tickets through the shadow slot:

`nw_triage_shadow_total` agree 15, disagree 5, every one served P0, shadow P1.

backtest, test split	Macro-F1	P0 recall
served, 0.05	0.663	0.903
candidate, 0.48	0.716	0.677

Agreement 0.953. `GATE FAILED`.

The retraining job exists, is paused, and never promotes

Track	The weekly job
AWS	EventBridge rule <code>northwind-<handle>-retrain-triage</code> , disabled
Google Cloud	Cloud Scheduler job, paused
Azure	Azure Machine Learning schedule, disabled
Local	<code>make pipeline-submit</code> by hand

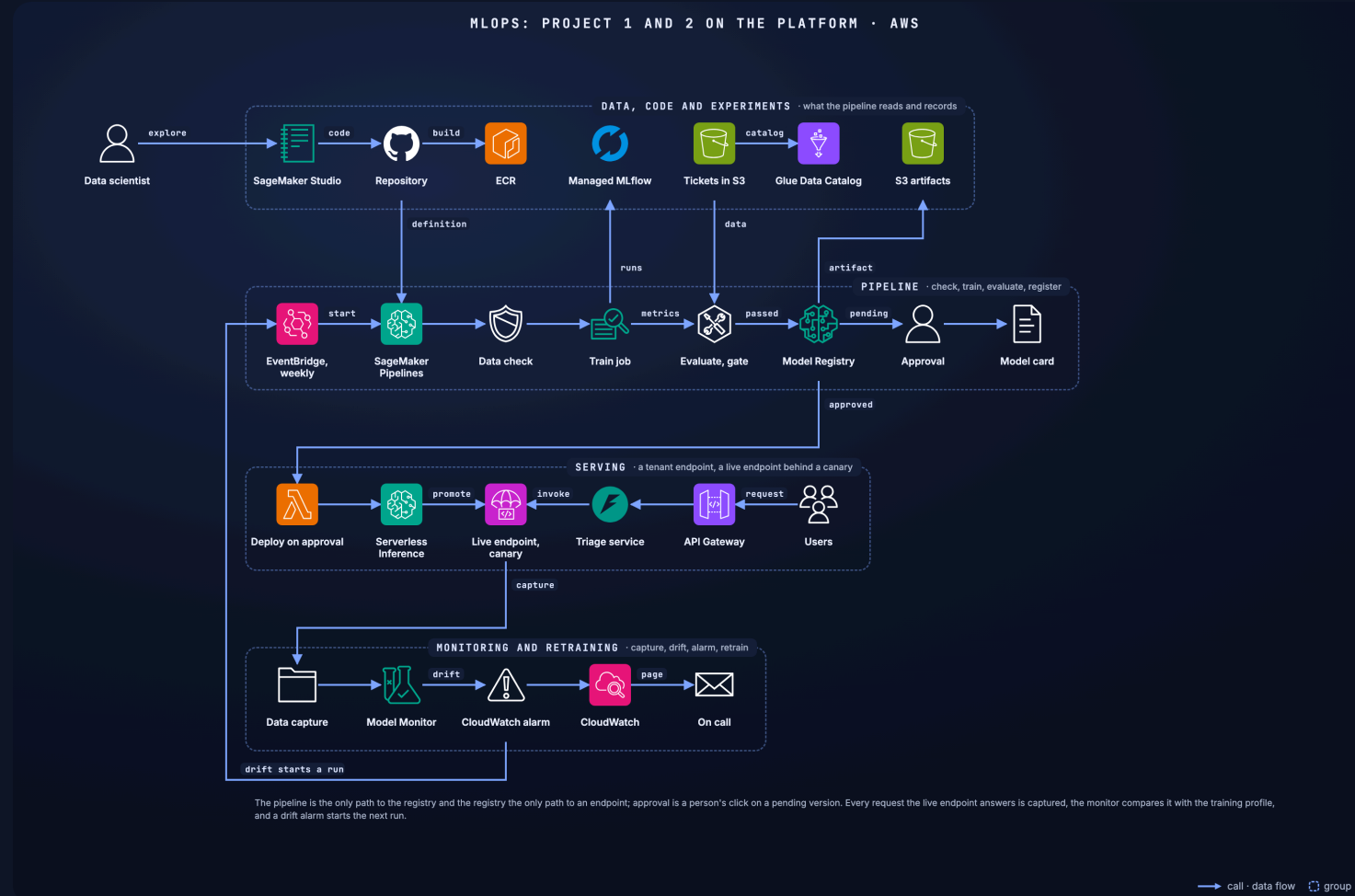
Every gate bar is a pipeline parameter: `ARGS="--min-p0-recall 0.9"` still registers at 0.903; 0.95 is refused and nothing is registered.

Two places define green

`retrain-triage.yml` runs the gate on pull requests against the committed summary and never registers anything.

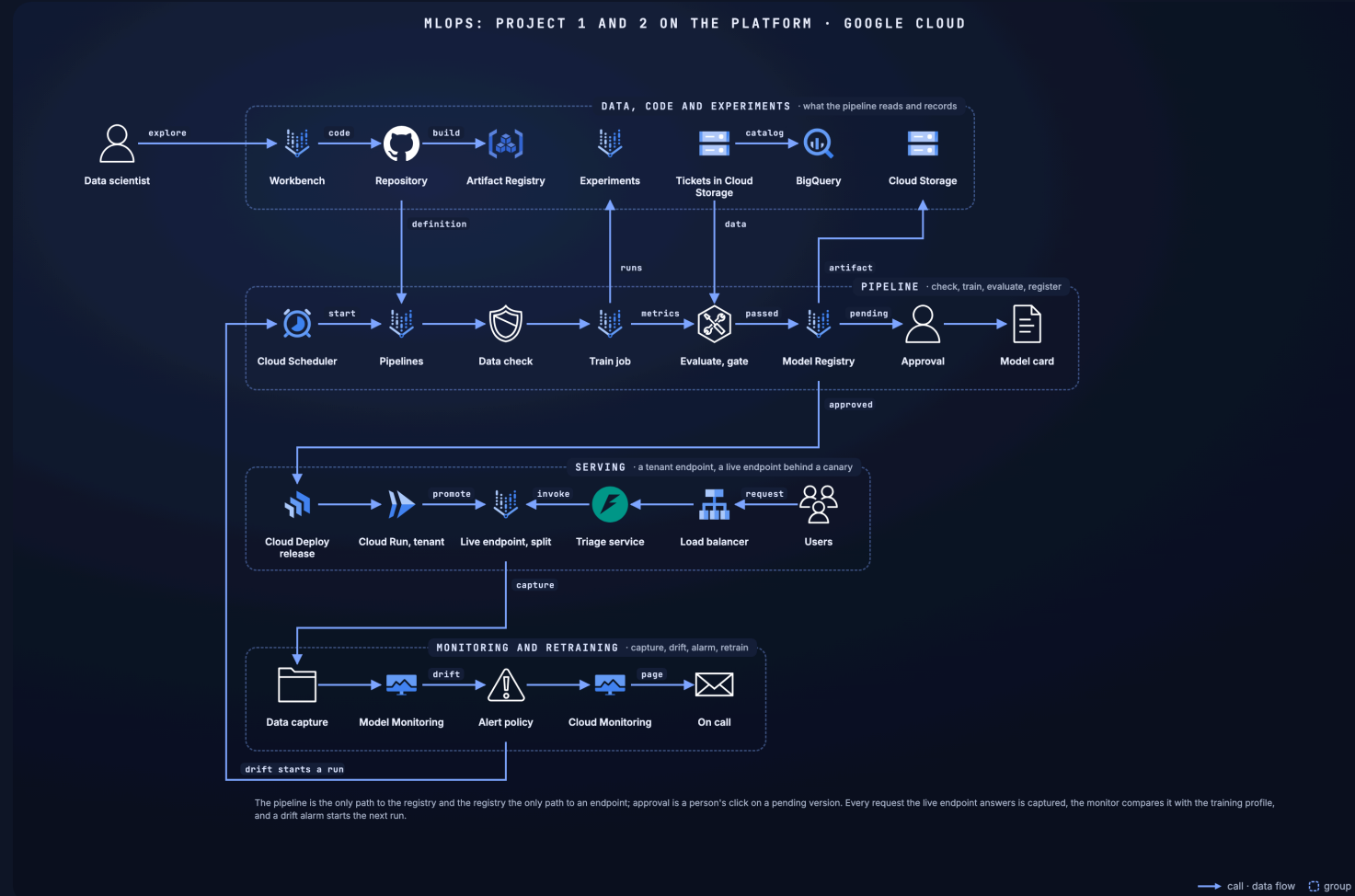
The registry approval is the other. Both read the same gate.

The MLOps loop, AWS track



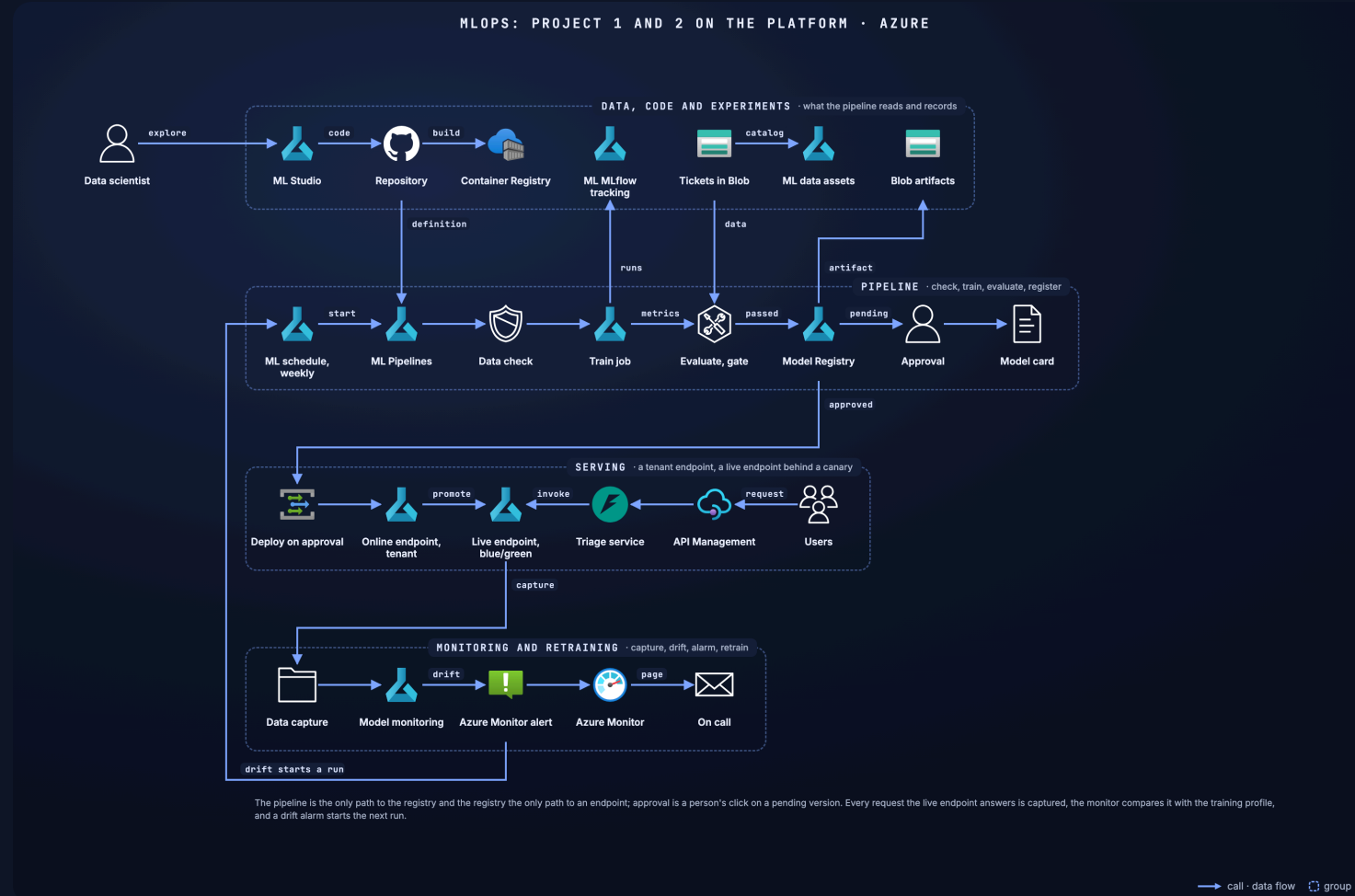
SageMaker Pipelines checks, trains, gates and registers; a person approves in the Model Registry; the approval deploys; Model Monitor watches the live endpoint's capture and a drift alarm starts the next run.

The MLOps loop, Google Cloud track



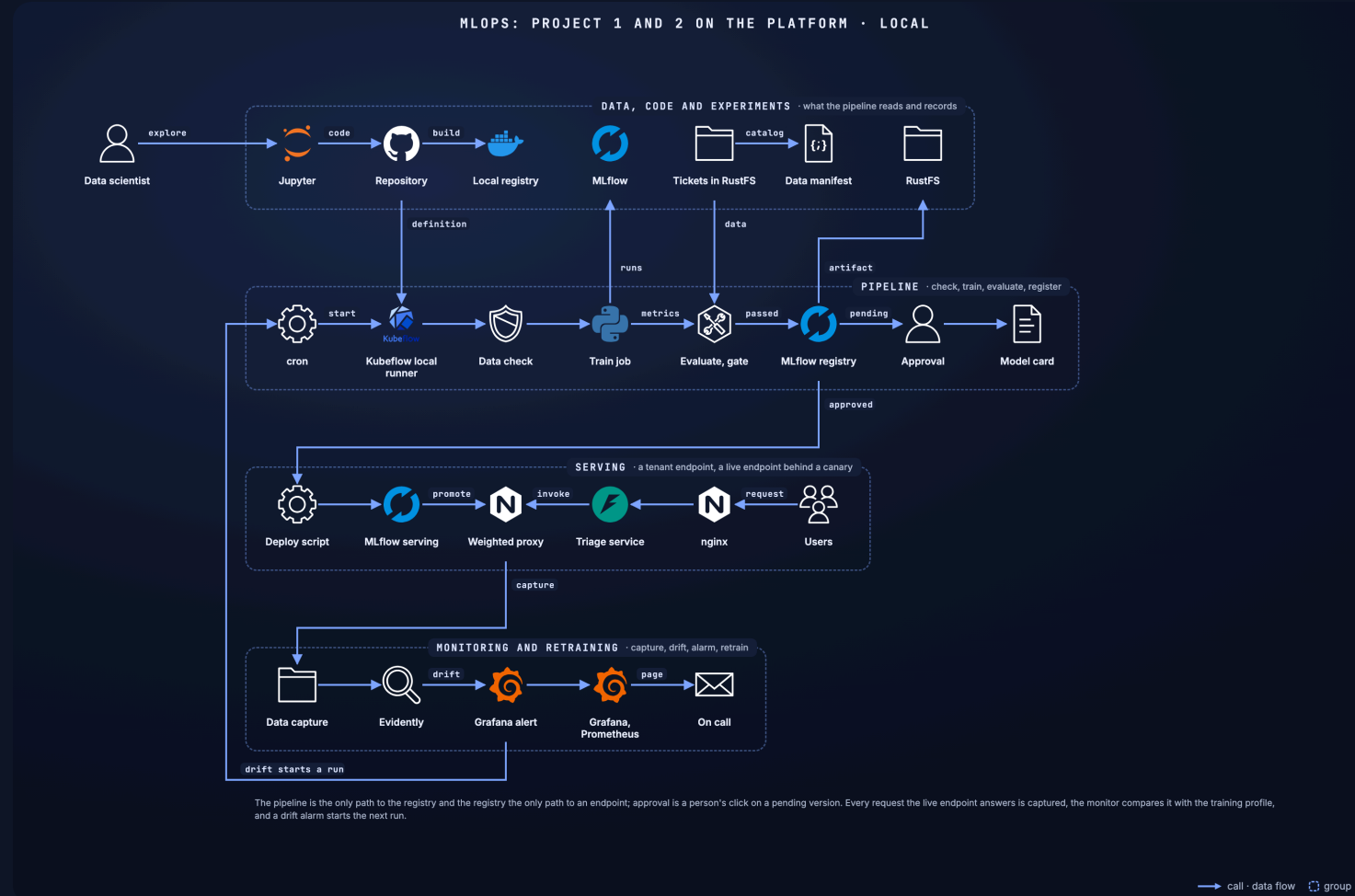
The same Kubeflow YAML on Pipelines on the Agent Platform (formerly Vertex AI); Model Registry with aliases, Cloud Run per tenant, a Vertex endpoint with a traffic split for live, Model Monitoring and an alert policy.

The MLOps loop, Azure track



Azure Machine Learning pipelines on serverless compute check, train, gate and register; the model registry carries the stage as a tag; a person approves; a managed online endpoint per tenant and a live endpoint with blue and green; model monitoring and an Azure Monitor alert start the next run.

The MLOps loop, Local track



The Kubeflow local runner, the MLflow registry with aliases, MLflow serving behind a weighted nginx proxy, Evidently and Grafana. One open-source service per managed box.

What exists after today, AWS track



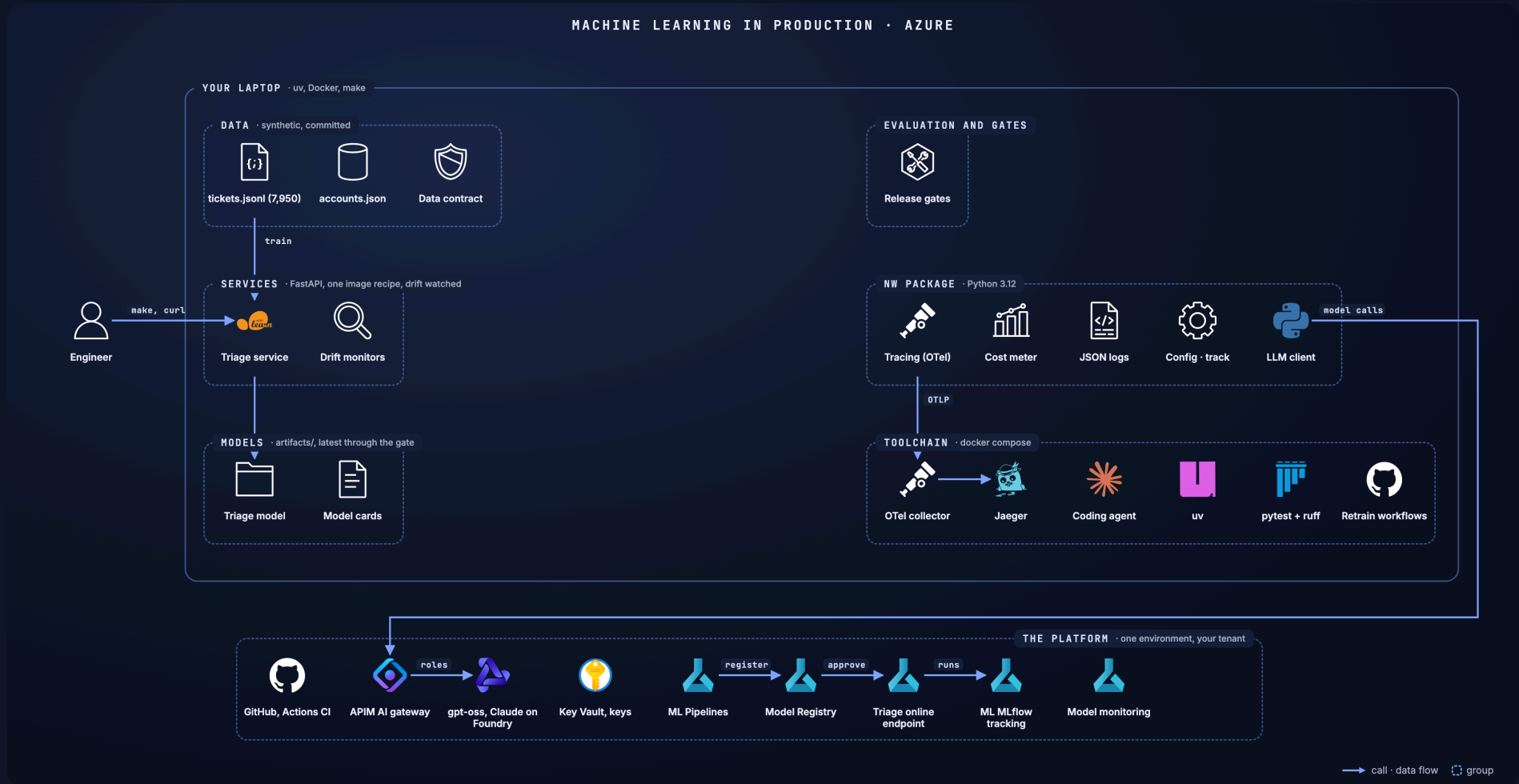
On the laptop: the data contract, the triage service, its drift monitors and model cards. On the platform: SageMaker Pipelines, the Model Registry, your serverless triage endpoint, managed MLflow and Model Monitor.

What exists after today, Google Cloud track



Same service and artifact; Pipelines, the Model Registry and your Cloud Run triage service reading the approved version.

What exists after today, Azure track



Same service and artifact; Azure Machine Learning pipelines and model registry, your managed online endpoint `nw-<handle>-trriage-<scope>` serving the approved version, MLflow tracking and model monitoring in the workspace.

What exists after today, Local track



Same service and artifact; the Kubeflow local runner, the MLflow registry and the serving tier on :8005 in the compose stack.

Acceptance criteria

1. The data passes its contract; the profile ships in the artifact
2. P0 recall at or above the bar by the threshold rule; the gate passed
3. The pipeline registered one candidate: 0.663 macro-F1, 0.903 P0 recall, 0.467 precision, 0.091 ECE
4. You approved it with a reason and your endpoint serves that version
5. The service: probes, metrics, correlation ID, drift, shadow

35 passed

what `make session02` prints when you are done

Questions worth asking

- Northwind's Enterprise P0 SLA is 15 minutes. What recall does that imply, and what precision can on-call absorb?
- The gate compares two models on one test split. What happens when the data changes, and who may force it?
- Your approval reason is on the registry for good. What would you want it to say in six months?
- What would change if the model became a transformer? What would stay?